

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	D. Gnana Deepthi	Reg.No.:	192411123
Department:	CSE	Date:	

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe

- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Gnanadeepthi

ANSWERS

1. I/O Communication Mechanism for a Real-Time Embedded System

Proposed Design

The system consists of multiple sensors connected to an **interrupt controller** and CPU.

Sensors → Interrupt Controller → CPU → ISR → Buffer → Main Application

Each sensor generates an interrupt every **1 ms**. A **vectored interrupt** is assigned to each sensor so that the CPU can directly execute the corresponding ISR.

Vectored Interrupt Handling

Each sensor has a separate interrupt vector:

Vector Sensor ISR

32 Sensor 1 ISR1

33 Sensor 2 ISR2

34 Sensor 3 ISR3

When a sensor generates an interrupt, the interrupt controller identifies it and directly transfers control to its ISR. This reduces polling overhead and improves response time.

ISR Operation

The ISR should be kept very short:

Sensor interrupt



Identify interrupt vector



Read sensor data



Store data in buffer



Clear interrupt



Return from ISR

Complex processing is performed later by the main application rather than inside the ISR.

Memory-Mapped vs I/O-Mapped I/O

Feature	Memory-Mapped I/O	I/O-Mapped I/O
Access	Normal load/store instructions	Special I/O instructions
Address space	Uses memory address space	Separate I/O space
Programming	Simple	More specialized
Latency	Low	Low

Selection: Memory-Mapped I/O

Memory-mapped I/O is selected because it provides simple and fast access to sensor registers using normal CPU instructions. It is well suited for real-time embedded systems where predictable and low-latency access is important.

CPU Utilization

Assume **10 sensors**, each generating **1000 interrupts/second**, and each ISR takes **10 μs**.

$$Total\ Interrupts = 10 \times 1000 = 10,000/sec$$

$$CPU\ Time = 10,000 \times 10\mu s = 0.1sec$$

$$CPU\ Utilization = 0.1 \times 100 = \boxed{10\%}$$

Since:

$$10\% < 15\%$$

the CPU utilization requirement is satisfied.

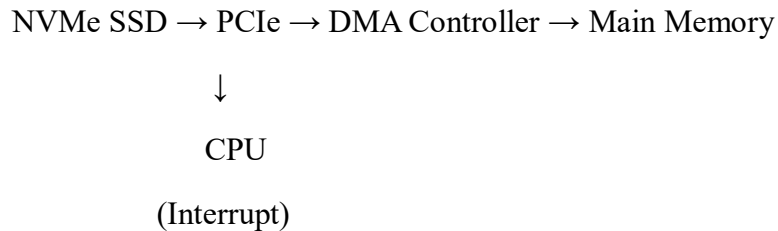
Conclusion

The proposed system uses vectored interrupts, memory-mapped I/O, short ISRs, and buffering to achieve low-latency sensor communication. With an estimated 10% CPU utilization, it satisfies the required less-than-15% CPU utilization constraint and is suitable for real-time embedded applications.

2. I/O Subsystem Design for High-Speed Storage

1. Proposed Design

The system uses NVMe over PCIe with a DMA controller to transfer data directly between storage and main memory.



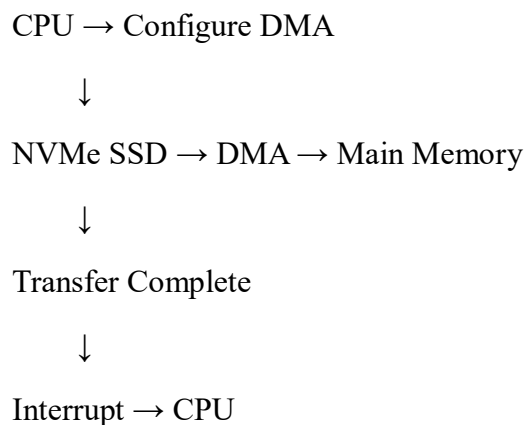
NVMe over PCIe

NVMe (Non-Volatile Memory Express) is selected because it is designed for high-speed SSDs and supports parallel I/O operations through PCIe.

The PCIe interface provides the required throughput of more than 3 GB/s.

DMA-Based Data Transfer

The CPU should not handle bulk data transfer. Instead, the **DMA controller** transfers data directly between the NVMe SSD and main memory.



This significantly reduces CPU workload and allows the CPU to perform other tasks while data is being transferred.

Interrupt Notification

After the DMA transfer is completed, the DMA controller generates an **interrupt** to notify the CPU.

The CPU then:

1. Checks the transfer status.
2. Verifies that the data transfer is complete.
3. Processes the received data or starts the next I/O operation.

Performance

The target throughput is:

$$\text{Throughput} > 3 \text{ GB/s}$$

Using NVMe + PCIe + DMA allows high-speed parallel data transfers while minimizing CPU involvement.

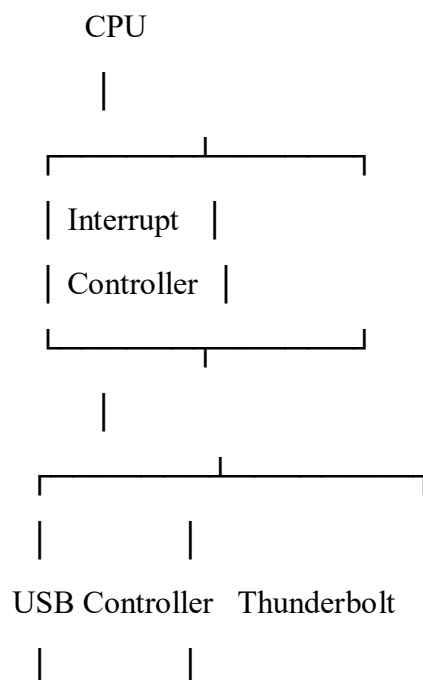
Conclusion

The proposed I/O subsystem uses NVMe over PCIe, DMA for bulk data transfer, and interrupt notification after completion. This achieves more than 3 GB/s throughput, reduces CPU overhead, and provides an efficient high-speed storage I/O system.

3. Multi-Device Communication Architecture

Proposed Design

The system contains **6 USB peripherals** and **2 high-speed Thunderbolt devices**.



6 USB Devices 2 High-Speed Devices

2. Interrupt Handling

Both **hardware and software interrupts** are used.

- **Hardware interrupts:** Generated by USB and Thunderbolt devices when data transfer or an important event occurs.
- **Software interrupts:** Used by the operating system to request device services or process deferred tasks.

To avoid **interrupt starvation**, interrupts are assigned suitable priorities and high-priority devices are prevented from continuously blocking lower-priority devices.

Fair Bus Arbitration

A **round-robin arbitration** scheme is used for USB devices so that each device gets a fair opportunity to access the bus.

For example:

USB1 → USB2 → USB3 → USB4 → USB5 → USB6 → USB1

Thunderbolt devices can use higher-speed PCIe-based transfers while the bus controller ensures that USB devices are also serviced regularly.

Avoiding Interrupt Starvation

- Use **interrupt priorities** carefully.
- Limit the time spent inside ISRs.
- Use **interrupt masking only when necessary**.
- Use buffering and deferred processing for lengthy operations.
- Apply **round-robin scheduling** for fair device access.

Conclusion

The architecture supports **6 USB and 2 Thunderbolt devices** using hardware and software interrupts. **Priority-based interrupt handling** prevents interrupt starvation, while **round-robin/fair arbitration** ensures that all peripherals receive regular bus access. This provides efficient and balanced multi-device communication.

4. Data Acquisition System Design

1. Proposed Architecture

The system continuously receives data at **500 MB/s** from a data acquisition device.

Data Acquisition Device



Memory-Mapped I/O



DMA Controller



Main Memory



CPU

2. Memory-Mapped I/O

The device registers are assigned specific memory addresses. The CPU can access these registers using normal memory read/write instructions.

This provides simple and fast access to device control and status registers.

3. DMA vs Interrupt-Driven I/O

Feature	DMA Transfer	Interrupt-Driven I/O
CPU involvement	Very low	High
Suitable for 500 MB/s	Yes	No
Data transfer	Directly to memory	CPU handles transfers
CPU overhead	Low	High
Efficiency	High	Lower

4. Selected Method: DMA

DMA is preferred because the data stream is continuous and very high-speed.

The CPU only configures the DMA controller initially. The DMA then transfers data directly from the device to memory without requiring the CPU for every data item.

After a block transfer is completed:

DMA Transfer Complete



Interrupt → CPU



Process received data

Conclusion

For a continuous 500 MB/s data stream, DMA with memory-mapped I/O is the best choice. It minimizes CPU intervention, provides efficient high-speed data transfer, and uses interrupts only to notify the CPU when a DMA block is completed.

5. Interrupt Handling Mechanism

Proposed Architecture

The system uses **nested and vectored interrupts** to achieve fast response.

High-Priority Device



Vectored Interrupt



Interrupt Controller



CPU



High-Priority ISR



Low-Priority ISR / Background Task

Nested Interrupts

Nested interrupts allow a high-priority interrupt to interrupt a lower-priority ISR.

For example:

Low-Priority ISR running



High-Priority interrupt occurs



Save current ISR context



Execute High-Priority ISR



Return to Low-Priority ISR

This ensures that critical devices are serviced immediately.

Vectored Interrupts

High-priority devices are assigned separate interrupt vectors. The CPU directly jumps to the required ISR without polling.

Device 1 → Vector 32 → ISR1

Device 2 → Vector 33 → ISR2

This reduces interrupt handling time and helps maintain the **latency below 10 μs**.

Software Interrupts

Software interrupts are used for **background tasks** such as:

- Data processing
- Buffer management
- Communication handling
- Non-critical system services

These tasks are given lower priority than hardware interrupts.

Latency Control

To maintain:

$$\text{Interrupt Latency} < 10 \mu s$$

the system should:

- Keep ISRs short.
- Give critical devices high priority.
- Use vectored interrupts.
- Avoid unnecessary interrupt disabling.
- Use nested interrupts for urgent events.
- Perform lengthy processing as background tasks.

Conclusion

The proposed mechanism uses nested interrupts, vectored interrupts for high-priority devices, and software interrupts for background tasks. Short ISRs and priority-based handling ensure that critical interrupts are serviced quickly, maintaining the required maximum latency of less than 10 μs .

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided